Bézier Curve: Generic (Any Degree, 2-D or 3-D)

GLMakie Visualisation

Code and Plot Explanation for Julia Beginners

`31-bezier-generic.jl`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

This document explains `31-bezier-generic.jl`. This is the most architecturally complete script in the series: a single program that handles *any degree* and *any dimension* (2-D or 3-D) by switching on the `CASE` constant. It introduces **GLMakie** (the interactive backend), a generic `BezierCurve` struct, a generalised De Casteljau loop, computed axis types (`Axis` vs `Axis3`), and auto-fitted plot limits.

Contents

1 The Two Plots **2**
1.1 Case 1: Quadratic 2-D 2
1.2 Case 2: Quartic 3-D 3

2 GLMakie: The Interactive Backend **3**

3 The Generic `BezierCurve` Struct **4**
3.1 Helper functions 4

4 Generic Math Functions **4**
4.1 `bernstein` — standalone basis function 4
4.2 `bz_point` — generic point via generator sum 4
4.3 `de_casteljau` — generalised while loop 5

5 `draw_and_save` — Branching on Dimension **6**
5.1 Conditional axis creation 6
5.2 Dynamic control point labels 6
5.3 Highlighted point labels: computed offsets 7
5.4 The De Casteljau drawing loop 7
5.5 Dynamic equation text 7
5.6 Auto-fitted 2-D limits 8

6 Type Map **8**

7 Entry Point **9**

1. The Two Plots

1.1. Case 1: Quadratic 2-D

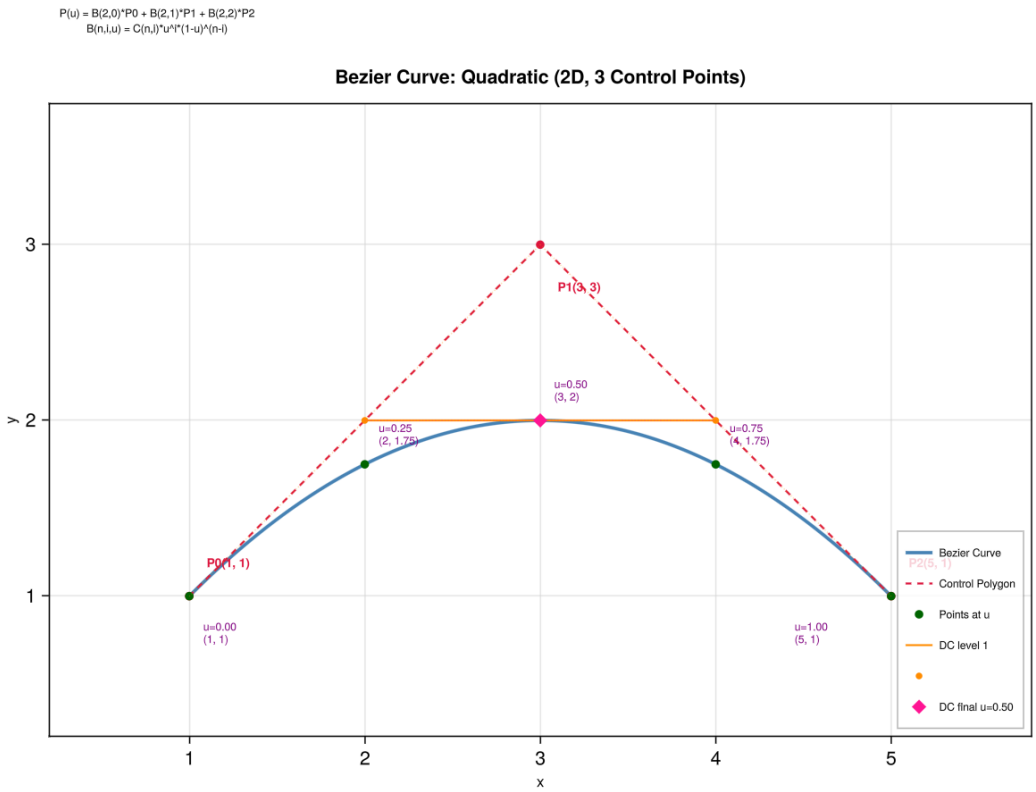


Figure 1: Case 1: quadratic Bézier in 2-D with De Casteljau construction at $u = 0.50$.

1.2. Case 2: Quartic 3-D

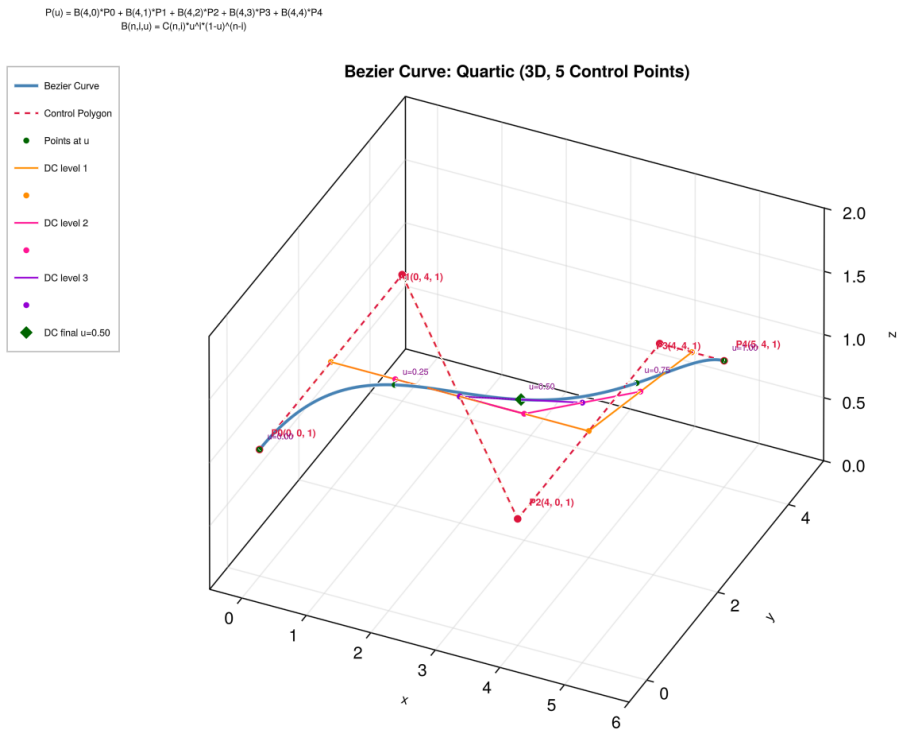


Figure 2: Case 2: quartic Bézier in 3-D with De Casteljau construction at $u = 0.50$. This plot is interactive when run with GLMakie — pan, zoom, and rotate with the mouse.

2. GLMakie: The Interactive Backend

All previous scripts used CairoMakie for static PNG output. This script switches to GLMakie — the interactive backend.

Backend	Output	Use case
`CairoMakie`	Static PNG, SVG, PDF	Publication figures
`GLMakie`	Interactive window (OpenGL)	Exploration, rotation, zoom
`WGLMakie`	Interactive browser (WebGL)	Jupyter notebooks

With GLMakie, the 3-D quartic plot can be rotated with the mouse, zoomed with the scroll wheel, and panned by dragging. This is why the note says *"the 3-D is actually can be panned, zoom, and rotate."* `save()` still works — GLMakie can save a static snapshot of the current view to PNG.

Install once with:

```
import Pkg; Pkg.add("GLMakie")
```

3. The Generic BezierCurve Struct

```julia
struct BezierCurve
    points ::Matrix{Float64}   # (n x dim)
    n      ::Int               # number of control points
    degree ::Int
    dim    ::Int               # 2 or 3
end

function BezierCurve(points::Matrix{Float64})
    n = size(points, 1)
    BezierCurve(points, n, n-1, size(points, 2))
end
```

This struct replaces all the specialised structs from previous scripts (`BezierQuadratic`, `BezierQuartic`, `DeCasteljauQuartic`, etc.) with one generic type.

`dim = size(points, 2)` reads the number of columns — 2 for a 2-D curve, 3 for a 3-D curve. `degree = n - 1` is always one less than the number of control points.

The constructor derives all four fields from the matrix alone. The caller only provides `points` and the struct figures out everything else.

> **Case 1 (quadratic 2-D):** `points` is $3 \times 2 \Rightarrow$ `n=3, degree=2, dim=2`
>
> **Case 2 (quartic 3-D):** `points` is $5 \times 3 \Rightarrow$ `n=5, degree=4, dim=3`

3.1. Helper functions

```julia
function degree_name(bz::BezierCurve)
    names = Dict(2=>"Quadratic", 3=>"Cubic", 4=>"Quartic", 5=>"Quintic")
    get(names, bz.degree, "Degree-$(bz.degree)")
end

dim_name(bz::BezierCurve) = bz.dim == 3 ? "3D" : "2D"
```

`degree_name` converts the degree integer to a human name, with a fallback `"Degree-N"` for unknown degrees. `dim_name` is a one-liner returning `"3D"` or `"2D"`.

Both are used to build the plot title dynamically:

```julia
title = "Bezier Curve: $(degree_name(bz)) ($(dim_name(bz)), $(bz.n)
↪  Control Points)"
```

This produces `"Bezier Curve:  Quadratic (2D, 3 Control Points)"` for Case 1 and `"Bezier Curve:  Quartic (3D, 5 Control Points)"` for Case 2 — without any `if/else`.

4. Generic Math Functions

4.1. bernstein — standalone basis function

```
function bernstein(n::Int, i::Int, u::Float64)
    binomial(n, i) * u^i * (1 - u)^(n - i)
end
```

A pure scalar function: given degree `n`, index `i`, and parameter `u`, it returns the i-th Bernstein basis value $\binom{n}{i} u^i (1-u)^{n-i}$. This is factored out as a named function (rather than inlined) because it is called inside a generator sum.

4.2. bz_point — generic point via generator sum

```
function bz_point(bz::BezierCurve, u::Float64)
    sum(bernstein(bz.degree, i-1, u) .* bz.points[i,:] for i in 1:bz.n)
end
```

This computes $P(u) = \sum_{i=0}^{n} B(n, i, u) \cdot P_i$ in a single `sum(generator)` expression.

For each `i` from 1 to `bz.n`, it computes `bernstein(...)` (a scalar) broadcast-multiplied by `bz.points[i,:]` (a length-`dim` row vector). The `sum` accumulates these vectors element-wise.

The result is a length-`dim` vector of `Float64` — either `[x, y]` or `[x, y, z]` depending on the curve's dimension. No `if is_3d` branching needed.

For Case 1 (quadratic 2-D) at $u = 0.5$:

`bz.degree = 2`, `bz.n = 3`

i	`bernstein(2, i-1, 0.5)`	`bz.points[i,:]`	product
1	$B(2, 0, 0.5) = 0.25$	$[1, 1]$	$[0.25, 0.25]$
2	$B(2, 1, 0.5) = 0.50$	$[3, 3]$	$[1.50, 1.50]$
3	$B(2, 2, 0.5) = 0.25$	$[5, 1]$	$[1.25, 0.25]$

Sum: $[0.25 + 1.50 + 1.25,\ 0.25 + 1.50 + 0.25] = [3.0, 2.0]$ ✓

4.3. de_casteljau — generalised while loop

```
function de_casteljau(bz::BezierCurve, u::Float64)
    levels  = Matrix{Float64}[copy(bz.points)]
    current = copy(bz.points)
    while size(current, 1) > 1
        n = size(current, 1)
        current = vcat([(
            (1-u) .* current[j,:] .+ u .* current[j+1,:]
        )' for j in 1:n-1]...)
        push!(levels, current)
    end
    return levels
```

```
end
```

This is the fully generalised De Casteljau algorithm. It replaces the fixed four-step `dc_steps` from script 22 with a `while` loop that runs until only one row remains.

`Matrix{Float64}[copy(bz.points)]`: Creates a `Vector{Matrix{Float64}}` — a vector whose elements are matrices — initialised with one element: a copy of the control point matrix. The type annotation `Matrix{Float64}[...]` is needed so Julia knows what type of vector to create.

`copy(bz.points)`: Makes an independent copy of the matrix. Without `copy`, modifying `current` would also modify `bz.points` because they would share the same memory.

The `while` loop: Each iteration applies one reduction step — identical to `reduce_level` from script 22 but written inline here. `push!(levels, current)` appends the new level matrix to the growing vector.

Return value: `levels` is a `Vector{Matrix{Float64}}` with `degree + 1` elements: `levels[1]` is L0 (n rows), `levels[2]` is L1 ($n-1$ rows), ..., `levels[end]` is the final level (1 row).

> **For Case 1 (quadratic) at $u = 0.5$:**
>
> Initial `current`: 3×2 matrix (L0).
>
> **Iteration 1:** `size(current,1) = 3 > 1`. Reduces to 2×2 (L1). `push!` appends to `levels`.
>
> **Iteration 2:** `size(current,1) = 2 > 1`. Reduces to 1×2 (L2 = final). `push!` appends.
>
> **Iteration 3:** `size(current,1) = 1`. Loop exits.
>
> `levels` has 3 elements: L0, L1, L2. `levels[end] = [[3.0, 2.0]]` (the curve point). ✓
>
> **For Case 2 (quartic) at $u = 0.5$, the loop runs 4 times** producing `levels` with 5 elements: L0 through L4.

5. `draw_and_save` — Branching on Dimension

This is the most branching-heavy function in the series. The `is_3d` flag controls whether to use `Axis3` or `Axis`, whether to pass two or three coordinate arguments to drawing functions, and how to compute label offsets.

5.1. Conditional axis creation

```
is_3d = bz.dim == 3
title = "Bezier Curve: $(degree_name(bz)) ..."

fig = Figure(size = (is_3d ? 900 : 800, is_3d ? 650 : 620), ...)
```

```
if is_3d
    ax = Axis3(fig[1,1], title=title, ..., elevation=..., azimuth=...)
else
    ax = Axis(fig[1,1], title=title, ...)
end
```

`is_3d` is a plain `Bool` variable computed once and reused throughout.

Ternary size: `is_3d ? 900 : 800` is Julia's ternary operator — evaluates to 900 if `is_3d` is true, 800 otherwise. The figure is slightly wider and taller for 3-D to give the `Axis3` room for its depth cues.

Both branches create a variable called `ax`. Julia's type system allows `ax` to be either an `Axis` or an `Axis3` — both are subtypes of `AbstractAxis` and both accept the same drawing functions (`lines!`, `scatter!`, `text!`).

5.2. Dynamic control point labels

```
for i in 1:bz.n
    pt     = cp[i,:]
    coords = join([@sprintf("%.4g", v) for v in pt], ", ")
    name   = "P$(i-1)($coords)"
    dy     = i % 2 == 1 ? 0.15 : -0.28
    ...
end
```

The `join` + `@sprintf` pattern: The list comprehension `[@sprintf("%.4g", v) for v in pt]` formats each coordinate with `"%.4g"` (up to 4 significant figures, dropping trailing zeros). `join(..., ", ")` concatenates the results with commas. For `pt = [0.0, 4.0, 1.0]`, this produces `"0, 4, 1"`. This works for both 2-D and 3-D without any branching — the comprehension adapts to however many coordinates the point has.

`dy = i % 2 == 1 ? 0.15 : -0.28`: Alternates the vertical offset between positive (above the point) and negative (below) based on whether the index is odd or even. This staggers consecutive labels so they don't overlap, without needing a hand-crafted offset array.

Labels generated for Case 1:

i	pt	coords	name
1	[1,1]	"1, 1"	"P0(1, 1)"
2	[3,3]	"3, 3"	"P1(3, 3)"
3	[5,1]	"5, 1"	"P2(5, 1)"

`dy` alternates: $i = 1$ (odd) → `0.15`; $i = 2$ (even) → `-0.28`; $i = 3$ (odd) → `0.15`.

5.3. Highlighted point labels: computed offsets

```
for (i, u) in enumerate(u_highlight)
    pt     = hpts[i,:]
    coords = join([@sprintf("%.3g", v) for v in pt], ", ")
    lbl    = @sprintf("u=%.2f\n(%s)", u, coords)
    dx     = u == 1.0 ? -0.55 : 0.08
    dy     = u in (0.0, 1.0) ? -0.28 : 0.10
    text!(ax, pt[1]+dx, pt[2]+dy, ...)
end
```

The 2-D label offsets are computed from the u value using two conditions:

`dx`: `-0.55` at $u = 1$ (shift left so the label doesn't run off the right edge), `0.08` everywhere else.

`dy`: `-0.28` at $u = 0$ and $u = 1$ (shift below, where there is space), `0.10` everywhere else (shift above).

`u in (0.0, 1.0)` uses Julia's `in` operator to check membership in a tuple — a compact alternative to `u == 0.0 || u == 1.0`.

5.4. The De Casteljau drawing loop

```
levels = de_casteljau(bz, u_dc)
for lv in 2:length(levels)
    pts      = levels[lv]
    col      = COLOR_DC[min(lv-1, length(COLOR_DC))]
    is_final = (lv == length(levels))
    lbl      = is_final ? @sprintf("DC final u=%.2f", u_dc)
                        : "DC level $(lv-1)"

    if !is_final
        lines!(ax, pts[:,1], pts[:,2], [pts[:,3] if is_3d]..., ...)
    end
    scatter!(ax, ...,
        marker = is_final ? :diamond : :circle,
        label  = is_final ? lbl : "")

    # Vertical connectors (interpolation triangle edges)
    prev = levels[lv-1]
    for j in 1:size(pts,1)
        for k in [j, j+1]
            seg_x = [prev[k,1], pts[j,1]]
            seg_y = [prev[k,2], pts[j,2]]
            lines!(ax, seg_x, seg_y, ...,
                color=(col, 0.35), linewidth=0.8, linestyle=:dot)
        end
    end
end
```

`COLOR_DC[min(lv-1, length(COLOR_DC))]`: `COLOR_DC = [:darkorange, :deeppink, :darkviolet, :darkgreen]`. Index `lv-1` starts at 1 (for L1) and goes up to `degree` (for the final level). `min(lv-1, length(COLOR_DC))` clamps the index to the length

of `COLOR_DC`, preventing an out-of-bounds error for curves of degree higher than 4. For a quartic, `lv-1` goes 1, 2, 3, 4 — exactly the four colours available.

`label = is_final ? lbl : ""`: Only the final level point (the diamond) gets a legend entry. All intermediate level lines and dots use `label = ""` to suppress their legend entries.

Vertical connectors: For each point `pts[j]` in the current level, two dotted lines are drawn: one from `prev[j]` to `pts[j]`, and one from `prev[j+1]` to `pts[j]`. These are the two edges of the interpolation triangle for that point — visually showing *which two parent points* each child point was interpolated from. The `(col, 0.35)` alpha makes them semi-transparent so they don't dominate the plot.

> **Connector drawing for Case 1 (quadratic), L1 at $u = 0.5$:**
>
> L0 (`prev`) has 3 rows: P_0, P_1, P_2. L1 (`pts`) has 2 rows: Q_0, Q_1.
>
> For $j = 1$ (Q_0): draws `prev[1]` → `pts[1]` and `prev[2]` → `pts[1]`. That is, two dotted lines converging at Q_0 from P_0 and P_1.
>
> For $j = 2$ (Q_1): draws `prev[2]` → `pts[2]` and `prev[3]` → `pts[2]`. Two dotted lines converging at Q_1 from P_1 and P_2.
>
> This creates the "V" shapes visible on the quadratic plot showing the interpolation ancestry of each level point.

5.5. Dynamic equation text

```
n     = bz.degree
terms = join(["B($n,$(i-1))*P$(i-1)" for i in 1:bz.n], " + ")
eq    = "P(u) = $terms\nB(n,i,u) = C(n,i)*u^i*(1-u)^(n-i)"
Label(fig[0, 1], eq, fontsize=8, font="Courier New", ...)
```

The equation string is built dynamically from the degree and number of control points.

For Case 1 (quadratic, 3 points): `"P(u) = B(2,0)*P0 + B(2,1)*P1 + B(2,2)*P2"`

For Case 2 (quartic, 5 points): `"P(u) = B(4,0)*P0 + ... + B(4,4)*P4"`

No hardcoded strings — the equation adapts to any degree automatically.

5.6. Auto-fitted 2-D limits

```
if !is_3d
    all_x = vcat(cp[:,1], cpts[:,1])
    all_y = vcat(cp[:,2], cpts[:,2])
    m = 0.8
    xlims!(ax, minimum(all_x)-m, maximum(all_x)+m)
    ylims!(ax, minimum(all_y)-m, maximum(all_y)+m)
end
```

For the 2-D case, instead of hardcoding `limits = (0.5, 6.0, 0.5, 3.5)`, the limits are computed from the actual data.

`vcat(cp[:,1], cpts[:,1])` concatenates the x coordinates of the control points and the 300 curve points into one vector. `minimum` and `maximum` find the extremes.

Adding/subtracting margin `m = 0.8` gives padding.

`xlims!(ax, xmin, xmax)` and `ylims!(ax, ymin, ymax)`: These Makie functions set the axis limits after the axis has already been created. They are the post-hoc equivalent of passing `limits = (...)` to `Axis(...)`.

This approach is necessary here because the limits depend on the data, which is only available after `bz_curve` has been called.

Auto-fitted limits for Case 1:

Control point x: min $= 1$, max $= 5$.

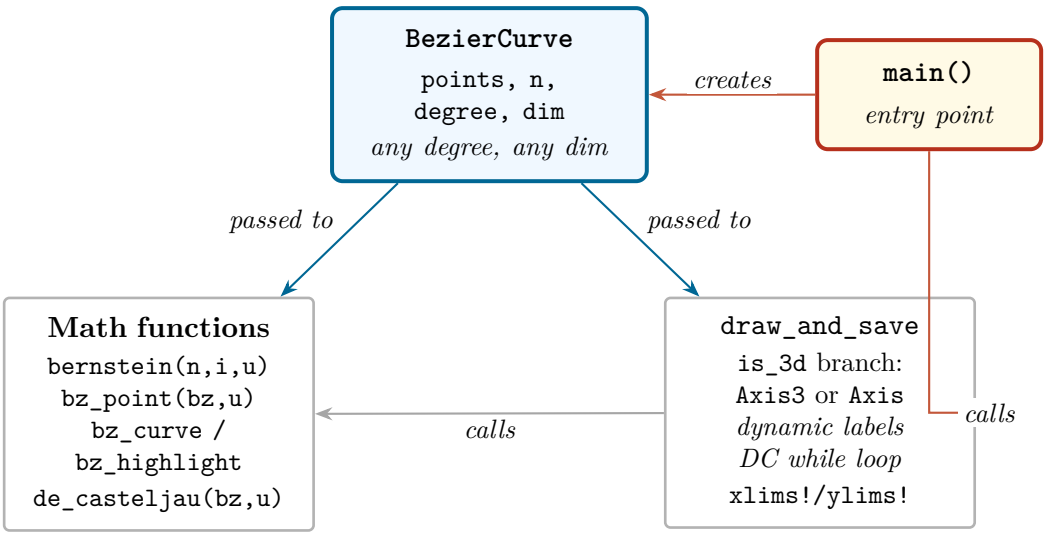
Curve x: min ≈ 1, max ≈ 5.

Combined: min $= 1$, max $= 5$.

With margin 0.8: `xlims!(ax, 0.2, 5.8)`.

Similarly y: control range $[1, 3]$, curve range $[1, 2]$, combined $[1, 3]$, with margin: `ylims!(ax, 0.2, 3.8)`.

6. Type Map



7. Entry Point

```
function main()
    data = CASE_DATA[CASE]
    bz   = BezierCurve(data.points)
    draw_and_save(bz, data.u_highlight, data.u_dc, data.output)
end

main()
```

`BezierCurve(data.points)` is the only constructor call. Everything else — degree, dimension, axis type, label format, limit computation — flows from the struct's fields automatically.

How to run:

```
julia 31-bezier-generic.jl
```

Change `const CASE = 1` or `= 2` at the top to switch between 2-D and 3-D. With GLMakie, the 3-D window is interactive: rotate with left-click drag, zoom with scroll, pan with right-click drag. Press Enter in the terminal to close.

Explanation for `31-bezier-generic.jl` | Directed and curated by E.R. Nurwijayadi | Code and explanations AI-generated